Made with Lpdf

The Quiet Revolution in Declarative Layout

A long-form article that overflows a single page and flows naturally onto the next.

For the better part of three decades, document layout on paper and document layout on screens evolved in parallel but separate worlds. Print systems — from TeX to InDesign — assumed a fixed, physical page and optimized for precise typography, consistent grids, and the delicate art of hyphenation. Web systems, meanwhile, grew up in a fluid medium where the viewport changes size, content reflows, and the designer cedes a great deal of control to the reader's device.

The tension between these two models is not merely aesthetic. It reveals a deeper disagreement about what a document is supposed to be. In the print tradition, a document is a finished artefact: a sequence of pages that look the same to every reader. In the web tradition, a document is a set of instructions that the browser interprets according to its own constraints. Neither model is wrong, but each struggles when asked to play in the other's territory.

The rise of declarative UI frameworks on the web — first React, then SwiftUI, then a long tail of smaller projects — demonstrated that fluid layout could be described concisely, composed predictably, and reasoned about without imperative escape hatches. Designers could say what they wanted, and a runtime would figure out how to make it work at any size. This was a genuine breakthrough, and it quietly changed the expectations of an entire generation of engineers. The question, then, was whether the same breakthrough could be brought back to print.

The answer turned out to be more subtle than it first appeared. A declarative layout language that paginates well cannot be a straightforward port of CSS Flexbox. Flexbox has no notion of a page break; its idioms assume an infinite vertical canvas. Porting them wholesale produces either clipped content, duplicated rendering, or worse — layouts that silently drop the overflow and never tell the author that anything went wrong. The honest alternative is to acknowledge pagination as a first-class concern and design the primitives around it.

In practice this means that each layout primitive needs two answers, not one: what does it look like when it fits, and what does it do when it doesn't. Some primitives have obvious answers. A vertical stack can split between children. A grid can split between rows. A paragraph of text can break between lines. Other primitives resist splitting by nature — a horizontal row of icons, for instance, cannot be cut in half vertically without becoming nonsense — and the right move is to accept them as atomic and let them move wholesale to the next page.

Frames fall into an interesting middle ground. A frame is typically used to represent a card, a callout box, or a labelled region, and cutting one in half across pages rarely produces a pleasing result. The cleanest rule turns out to be: treat the frame as atomic, and force any overflow to the next page. Authors who want the frame's *contents* to flow can use a stack instead. This gives them a clear choice between a bounded visual unit and an open flow region — a choice the layout engine can enforce mechanically rather than through convention.

The same reasoning applies to grid cells and cluster items. Within a single row of a grid, the cells are laid out side by side and reach a common height; splitting a single cell mid-content would produce inconsistent row heights and break the visual rhythm the author is trying to establish. The atomic-children rule keeps the grid honest: rows split between themselves, but never within a single row. Readers see complete rows on each page, and the grid feels like a grid, not an accidental flow.

Page numbering and running headers deserve a separate treatment. They are not part of the content flow — they are chrome, in the old Netscape sense of the word: fixed decoration that surrounds the document. The cleanest way to express this is a small annotation that marks a child as chrome, telling the paginator to render it identically on every output page and reserve its space from the flow budget.

Made with Lpdf

Combined with a simple substitution for **{page}** and **{pages}** tokens, this gives authors a letterhead, a continuation header, and a page-number footer with three extra attributes and no new primitives.

None of this is conceptually difficult, and none of it is trivial to get right. The subtleties come out in the long tail of cases: what happens when a chrome element is taller than a page; what happens when the only flow content on a page is itself an indivisible atomic block; what happens when a long text block ends exactly on a page boundary and a single trailing line would otherwise orphan itself. Each of these cases has a best answer; each of them has been debated at length by the few people in the world who care deeply about this kind of thing.

What ties the whole project together is a commitment to boring predictability. Authors should not have to read the implementation to understand what happens to their content at a page break. The engine should do the thing they expected, every time, and when it cannot — when the content fundamentally does not fit — it should produce a clear signal rather than silently dropping pixels on the floor. That, ultimately, is the revolution: not new primitives, but a new level of trust between author and engine.

A lot has been written about the technical underpinnings of these systems, and readers interested in the deeper mechanics will find entire books on the topic of hyphenation alone. This article does not aim to be exhaustive. Its purpose is simpler: to produce enough text, arranged on a single page, that the layout engine has no choice but to demonstrate its pagination behaviour. If you are reading this paragraph on the second page of the rendered PDF, the demonstration is working as intended.

— end of article —